

Generating Handwritten Digits using a simple Diffusion Model on MNIST Dataset*

IMAM ABDULRAHMAN BIN FAISAL UNIVERSITY COLLEGE OF CS and IT

Abstract—In this work, we apply a Denoising Diffusion Probabilistic Model (DDPM) to generate digits from the handwritten MNIST dataset. Diffusion models, popular for being used in all text-to-image models, and known for their stability and robust performance. By implementing a U-Net-based architecture for noise prediction and a simple noise schedule, we try using diffusion models for generating clear, different digit samples. The experiments highlight the model's ability to avoid mode collapse and produce consistent results across multiple epochs, what we did offers insights into the application of diffusion models to simple image generation tasks.

Index Terms—Denoising Diffusion Probabilistic Model (DDPM), Generative Modeling , Image Generation, MNIST Dataset, Noise Prediction

I. GENERATIVE MODELING OVERVIEW

Generative modeling is all about creating new data that looks similar to the data we have. For example, generating images of handwritten digits that look like images in the MNIST dataset. Some of the most popular ways to do this are through **Generative Adversarial Networks (GANs)**, **Variational Autoencoders (VAEs)**, and **Diffusion Models**. GANs, first introduced by Goodfellow et al. in 2014, work like a game where one network (the generator) tries to make fake images, and another network (the discriminator) tries to tell whether the images are real or fake. While GANs can produce very realistic images, they can also be difficult to train and sometimes fail by creating only a small variety of images. VAEs, proposed by Kingma and Welling in 2013, use a different approach where they learn to map data into a simpler form and then rebuild it. Although VAEs are easier to train, their generated images are often blurry. **Diffusion Models** are a newer approach that overcomes many of these problems. They add noise to the data and then gradually remove it step-by-step to create new samples. Diffusion models are better at producing high-quality images and have more stable training than GANs or VAEs.

II. METHODOLOGY AND IMPLEMENTATION

A. Maintaining the Integrity of the Specifications

Model Architecture : For generating handwritten digits, we implemented a **U-Net-based architecture**, which is a type of convolutional neural networks(CNNs) originally designed for image segmentation tasks. This model is particularly effective for image generation because it captures both high-level features and fine details. The U-Net consists of 2 main parts:

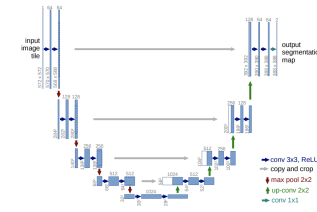


Fig. 1. U-net architecture (example for 32x32 pixels in the lowest resolution). Each blue box corresponds to a multi-channel feature map. The number of channels is denoted on top of the box. The x-y-size is provided at the lower left edge of the box. White boxes represent copied feature maps. The arrows denote the different operations.

Fig. 1. U-Net

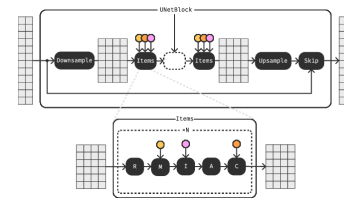


Fig. 2. U-Net Architecture with Decoder and Encoder Overview

- **Encoder (Contracting Path):** This part captures context by downsampling the input image, extracting essential features.
- **Decoder (Expansive Path):** This part enables precise localization by upsampling the feature maps and combining them with corresponding feature maps from the encoder through skip connections.[https://en.wikipedia.org/wiki/U-Net?Wikipedia link](https://en.wikipedia.org/wiki/U-Net?Wikipedia%20link)

In our implementation, the model predicts the noise added to an image at each diffusion step, which is then used to reconstruct the original image during the reverse diffusion process.

a) *Diffusion Process:* The diffusion process involves two main phases:

- **Forward Diffusion (Noising):** Starting with a clean image, we gradually add Gaussian noise over several steps, transforming the image into pure noise.
- **Reverse Diffusion (Denoising):** We train the model to reverse this process by learning to remove the added noise step-by-step, reconstructing the original image.

Mathematically, at each step t , the noisy image $x(t)$ is generated by adding noise to the previous image $x(t-1)$, controlled by a variance schedule t . The model is trained to predict the noise added at each step, enabling it to reverse the process during generation

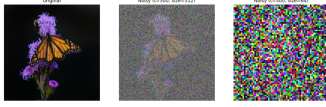


Fig. 3. Denoising Process Illustration

Algorithm 2 Sampling

```

1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2: for  $t = T, \dots, 1$  do
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$ 
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\alpha_t}} \mathbf{z}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 
5: end for
6: return  $\mathbf{x}_0$ 

```

Fig. 4. Mathematical Algorithm behind sampling

b) Noise Schedule and Beta Schedule: The **beta schedule** defines how much noise is added at each diffusion step. A common approach is to use a linear schedule, where t increases linearly from a small value (e.g., 0.0001) to a larger value (e.g., 0.02) over the total number of steps T . This schedule controls the variance of the noise added at each step, influencing the model's ability to learn and generate high-quality images.

c) Training Setup: The MNIST dataset was used for training, consisting of 28x28 grayscale images of handwritten digits. The following training setup was employed:

Data Preprocessing: The images were normalized to have pixel values between -1 and 1 to ensure stable training.

Loss Function: The model was trained to minimize Mean Squared Error (MSE) between the predicted noise and the actual noise added during the forward diffusion process.

Optimizer: We used the Adam optimizer with a learning rate of 0.001 to update the model weights.

Training Duration: Initially, we attempted to train the model for only 5 epochs, but the results were suboptimal. The images generated were not very clear, and the diversity of generated digits was limited. After extending the training to 15 epochs, the results improved significantly. However, the model's performance plateaued around this point, showing minimal improvement with further training. This suggests that while the model was learning, it reached its capacity after 15 epochs under the current setup.

GPU Limitations: Due to computational constraints and limited GPU memory, we were unable to add more layers to the model or further extend the training duration. This limited the model's ability to learn more complex features and prevented the model from fully exploiting its potential.

Implementation Details To optimize for the MNIST dataset and manage computational limitations:

Simplified Architecture: We reduced the number of layers in the U-Net architecture to ensure the model could run within the memory constraints of the GPU. While this choice helped avoid memory issues, it also limited the model's capacity to learn complex representations.

Time Embedding: The time step

It was encoded and added as an extra input channel, allowing the model to condition its predictions based on the

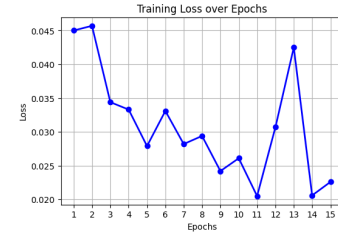


Fig. 5. Loss Curve during the 15 epochs

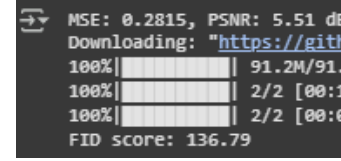


Fig. 6. Metrics Results

specific diffusion step. This helped the model better understand the denoising process at each stage.

Skip Connections: Skip connections between corresponding encoder and decoder layers were included to preserve spatial information, improving the quality of the generated images.

This approach allowed us to generate reasonably high-quality handwritten digits while staying within the limits of available computational resources.

III. EXPERIMENTAL RESULTS AND ANALYSIS - DATASET AND PREPROCESSING

We evaluate the quality of our generated digits using a few important metrics:

- **Preprocessing:** We first convert the pixel values of the generated and real MNIST images from the range $[-1, 1]$ to $[0, 1]$ so they can be compared properly.
- **MSE & PSNR:** Next, we calculate two metrics: **Mean Squared Error (MSE)**, which measures how much the generated images differ from the real ones, and **Peak Signal-to-Noise Ratio (PSNR)**, which helps us determine how clear and detailed the generated images are.
- **FID:** We also compute the **Fréchet Inception Distance (FID)**, which compares the statistical similarity between the real and generated images. A lower FID means the generated images are more similar to the real ones.

These metrics give us a clear picture of how well our model is generating realistic digits

DISCUSSION OF IMPLICATIONS AND FUTURE WORK

This section looks at how well the diffusion model worked, its limitations, and where we can take this research in the future.

Implications: Our diffusion model worked well overall, generating high-quality and diverse handwritten digits. It was more stable during training than GANs, which often have trouble with unstable training and mode collapse.

Challenges and Limitations: Despite the good performance, the model wasn't perfect. Some digits were harder to generate than others, and we noticed some visual flaws in the images. These issues show there's still room for improvement.

Future Work: Going forward, we could apply the diffusion model to more complex tasks, such as generating higher-resolution images. We might also work on making the model more efficient or add features like class conditioning to generate specific digits with more control.

Broader Impact: Diffusion models are promising for many different applications. They could be used in various fields, including generating realistic art or even in industries like healthcare, where generating medical images could be useful.

CONCLUSION AND REFERENCES

In this study, we explored the use of Denoising Diffusion Probabilistic Models (DDPMs) to generate handwritten digits from the MNIST dataset. Our implementation utilized a simplified U-Net architecture and a linear noise schedule, focusing on training stability and image quality.

Key Findings Training Stability: The diffusion model demonstrated stable training without issues like mode collapse, which are common in Generative Adversarial Networks (GANs).

Image Quality: Generated digits were clear and diverse, with evaluation metrics indicating satisfactory performance.

Evaluation Metrics: We employed several metrics to assess the quality of the generated images:

Mean Squared Error (MSE): Quantifies the average squared difference between the original and generated images. PMC

Peak Signal-to-Noise Ratio (PSNR): Measures the ratio between the maximum possible power of a signal and the power of corrupting noise.

Fréchet Inception Distance (FID): Assesses the similarity between the distribution of generated images and real images.

Limitations and Future Work While the model performed well, there are areas for improvement:

Digit Variability: Some digits, like '4' and '5', exhibited higher variability in handwriting styles, which the model found challenging to generate consistently.

Visual Artifacts: Minor artifacts were present in some generated images, indicating room for refinement in the model's architecture or training process.

Future directions include:

Higher-Resolution Datasets: Applying the model to more complex datasets to test its scalability and adaptability.

Model Optimization: Enhancing the model's efficiency and performance through architectural adjustments or advanced training techniques.

Class Conditioning: Implementing class conditioning to generate specific digits or characters, improving control over the generated outputs.

This research contributes to the understanding of diffusion models in image generation tasks, highlighting their potential and areas for further exploration.

References

Ho, J., Jain, A., Abbeel, P. (2020). Denoising Diffusion Probabilistic Models. NeurIPS. Retrieved from

Wikipedia contributors. (2024). Fréchet Inception Distance. Wikipedia. Retrieved from

Wikipedia contributors. (2024). Peak Signal-to-Noise Ratio. Wikipedia. Retrieved from

Beigy, H. (2023). Evaluating Generative Models. Sharif University. Retrieved from